

Note on gpt.py computational workflow.

- RMS norm: normalization of token's embedding vector $\mathbf{x} \in \mathbb{R}^d$. This is different from transformer paper's LayerNorm.

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x}) + \varepsilon}, \quad \text{RMS}(\mathbf{x}) = \sqrt{\sum_{i=1}^d x_i^2}, \quad \mathbf{x} = (x_1, \dots, x_d), \quad \varepsilon > 0.$$

- Rotary Positional Embedding (RoPE): positional embedding using sin, cos positional tensors. This is different from transformer paper's Positional Embedding.

Input x with shape (B, T, H, D) where B =batch size, T =sequence length, H =number of heads in multi-head attention, D =dimension of each head (must be even). Let $\sin = (\sin^{t,i})_{0 \leq t \leq T-1, 0 \leq i \leq D/2}$, $\cos = (\cos^{t,i})_{0 \leq t \leq T-1, 0 \leq i \leq D/2}$ be tensors each with shape $(1, T, 1, D/2)$. Then for each batch b , each index $t \in \{0, \dots, T-1\}$, each head h , RoPE does the following:

$$(x^{b,t,h,0}, \dots, x^{b,t,h,D-1}) \rightarrow \begin{cases} \cos^{t,i} x^{b,t,h,i} + \sin^{t,i} x^{b,t,h,i+D/2} & \text{for } 0 \leq i \leq D/2 - 1, \\ -\sin^{t,i-D/2} x^{b,t,h,i-D/2} + \cos^{t,i-D/2} x^{b,t,h,i} & \text{for } D/2 \leq i \leq D-1. \end{cases}$$

Position information, originally in t , is encoded in such rotation via $\cos, \sin$. Usually we take $\cos^{t,i} = \cos\left(\frac{t}{10000^{2i/D}}\right)$ and $\sin^{t,i} = \sin\left(\frac{t}{10000^{2i/D}}\right)$.

- Causal Self-Attention. The self-attention module with specific causal mask construction.

- number of heads `n_head % number of kv heads n_kv_head = 0`.

- self-attention. Input x has shape $(B, T, _)$, which stands for the current forward pass. Then Q, K, V are from input x , i.e., $Q = W_Q^T X$ etc. They are re-shaped into shape $(B, T, \text{n_head}, \text{head_dim})$ for Q and $(B, T, \text{n_kv_head}, \text{head_dim})$ for K, V , respectively. Then apply RoPE to Q, K , apply RMSNorm to Q, K , finally apply $(1, 2)$ transpose $(B, T, H, D) \rightarrow (B, H, T, D)$ to Q, K, V , i.e., make head be batch dim, so that we can parallel process each attention head to the full batch;

- kv-cache and multi-query-attention. T in (B, H, T, D) stands for the number of queries. For Q , use number of queries in this forward pass = T_q ; for K, V , after kv-cache, use number of queries in the cache + current forward pass = $T_q + (T_k)_{\text{prev}} = T_k$.

PyTorch 2.0's `scaled_product_attention`:

```
def scaled_dot_product_attention(
    query: Tensor,
    key: Tensor,
    value: Tensor,
```

```

    attn_mask: Tensor | None = None,
    dropout_p: float = 0.0,
    is_causal: bool = False,
    scale: float | None = None,
    enable_gqa: bool = False,
) -> Tensor: ...

```

Multi-Query-Attention (Group-Query-Attention, gqa):

`enable_gqa = self.n_head != self.n_kv_head`: multiple query heads use the same key/value head, or in other words duplicately use key/value head for a chunk of query heads. This saves computation. Example: `n_head= 16`, `n_kv_head= 4`, then each group of 4 query heads share the same kv head, or the same kv head repeatedly used 4 times for each chunk of 4 query heads.

Mask construction in multi-query-attention: Discuss the following cases

- (1) Inference with no increment, i.e. infer the whole sequence at once ($T_q = T_k$) or training (kv cache is empty): Auto regressive case. Each query only attends keys/values before it. Use standard causal mask. See Figure 1 left.
- (2) $T_q = 1$, inference with single query in this forward pass. Each query attends all the keys/values in the cache. No mask needed. See Figure 1 middle.
- (3) Not (1) or (2): $1 < T_q < T_k$. Inference with a chunk of T_q queries in this forward pass. Each query attends all the keys/values in the old kv cache $(T_k)_{\text{prev}} = T_k - T_q$, but for the newly added T_q items of kv, i.e. within the chunk we use causal attention so that each query only attends keys/values before it. See Figure 1 right.

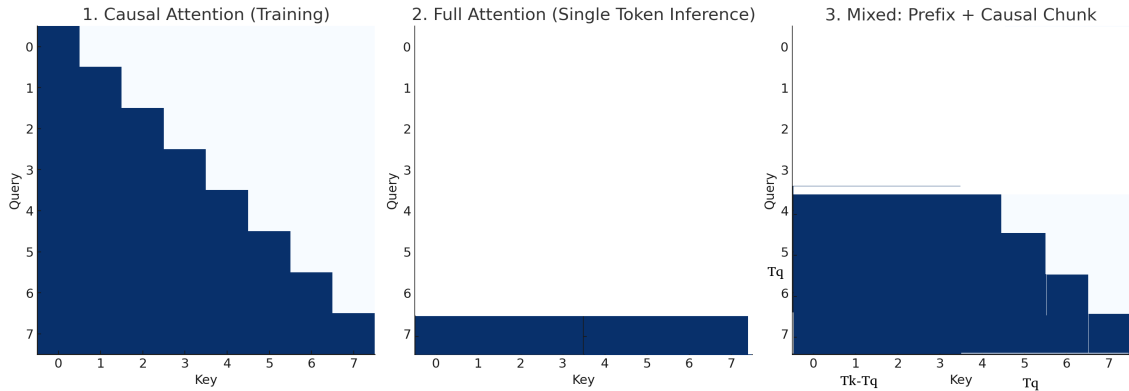


Figure 1: MQA masks.

– final projection. Concatenate heads

$$(B, \text{n_head}, T, \text{head_dim}) \rightarrow (B, T, \text{n_head}, \text{head_dim}) \rightarrow (B, T, \text{n_head} \times \text{head_dim}) .$$

Followed by a linear layer $\text{n_head} \times \text{head_dim} \rightarrow \text{n_embd}$.